

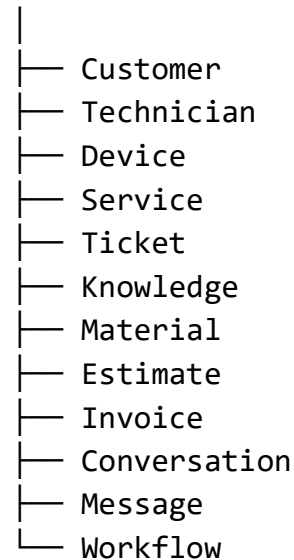
BiteFixes Enterprise Reference Model (BERM)

Principio 1: Todo es una Entidad de Negocio

No pensamos primero en tablas ni APIs.

Pensamos en entidades.

Company



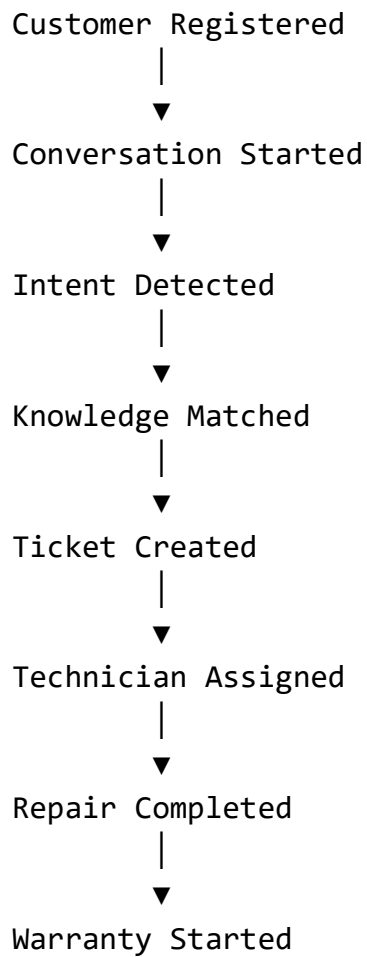
Estas son las piezas fundamentales del sistema.

Principio 2: Todo cambia mediante Eventos

Las entidades no cambian "porque sí".

Cambian cuando ocurre un evento.

Ejemplo:



Así toda la historia de una empresa puede reconstruirse.

Principio 3: Todo tiene un Estado

Ejemplo:

Ticket

Draft

↓

Open



Diagnosis



Waiting Parts



Repair



Testing



Ready



Delivered



Closed

Nunca debería existir un ticket en un estado inválido.

Principio 4: Todo tiene un Ciclo de Vida

Customer

Prospect



First Contact



Active



VIP



Inactive



Archived

Device

Registered



Received



Diagnosis



Repair



Delivered



History

Principio 5: Todo pertenece a una Empresa

Nunca existe un objeto "global".

Siempre existe un contexto.

Company



Customer



Conversation



Ticket



Device

↓

Knowledge

Esto simplifica el modelo multiempresa (multi-tenant).

Principio 6: Todo deja una Huella

Cada cambio importante genera un registro de auditoría.

Ejemplo:

Timestamp

↓

User

↓

Module

↓

Operation

↓

Previous Value

↓

New Value

No solo por seguridad, sino también para soporte y análisis.

Principio 7: Nada está Acoplado

En lugar de:

Ticket



Customer Service

La comunicación pasa por eventos.

Ticket Opened



Notification



Analytics



Inventory



Bitey



Timeline

Cada componente es independiente.

Principio 8: El Dominio manda

La interfaz nunca define las reglas.

Por ejemplo:

La aplicación móvil puede desaparecer.

WhatsApp puede cambiar.

FastAPI puede reemplazarse.

Pero el dominio sigue siendo el mismo.

Principio 9: Los Motores son Especialistas

En vez de un único "motor de IA", Bity tendrá varios motores especializados.

Language Engine

Intent Engine

Knowledge Engine

Context Engine

Planning Engine

Decision Engine

Recommendation Engine

Translation Engine

Workflow Engine

Learning Engine

Cada uno tiene una única responsabilidad.

Principio 10: La IA no decide sola

Bitey propone.

Las políticas validan.

Los flujos ejecutan.

Por ejemplo:

Customer asks for repair

↓

Intent Engine

↓

Knowledge Engine

↓

Decision Proposal

↓

Policy Engine

↓

Workflow Engine

↓

Execution

Así evitamos comportamientos impredecibles.

Principio 11: Todo es Configurable

Nada importante debe estar codificado.

Ejemplos:

- Horarios.
- Garantías.
- Estados.
- Prioridades.
- Idiomas.
- Reglas.
- Flujos.
- Integraciones.

Todo debe vivir en configuración o base de datos.

Principio 12: El Backend es una Plataforma

No lo vería como una API REST.

Lo vería como una plataforma que ofrece capacidades.

Por ejemplo:

Customer Capability

↓

Conversation Capability



Ticket Capability



Knowledge Capability



Inventory Capability



Analytics Capability

Las APIs son solo una forma de acceder a esas capacidades.

Principio 13: Preparado para IA Agéntica

Hoy Bitey responde.

Mañana podrá ejecutar objetivos.

Ejemplo:

Objetivo:

"Reducir el tiempo medio de reparación."

Bitey podría:

1. Analizar estadísticas.
2. Detectar cuellos de botella.
3. Identificar servicios lentos.

4. Recomendar mejoras.
5. Sugerir cambios en políticas.

Todo sin modificar directamente el negocio, sino generando propuestas para revisión.

Principio 14: Plataforma Extensible

En el futuro podrían existir extensiones oficiales.

Marketplace

↓

Inventory Pro

↓

Accounting

↓

Electronic Invoice

↓

Advanced AI

↓

BI Dashboard

↓

CRM Pro

↓

Call Center



Remote Support

Cada módulo debería poder instalarse sin alterar el núcleo.

Principio 15: Arquitectura para la próxima década

Si tuviera que resumir toda la filosofía de BiteFixes en una sola frase, sería esta:

El núcleo debe permanecer estable; todo lo demás debe poder evolucionar alrededor de él.

Eso implica que:

- La base de datos puede evolucionar mediante migraciones.
- Los motores de IA pueden cambiar de proveedor.
- Los canales (WhatsApp, Web, App) pueden aumentar.
- Los módulos pueden crecer.
- Las integraciones pueden reemplazarse.

Pero el **modelo de negocio** y la **arquitectura central de Bitey** deben mantenerse consistentes.

El siguiente gran paso

A partir de aquí dejaría de producir documentación conceptual y comenzaría la **especificación técnica ejecutable**. El objetivo sería construir un **BiteFixes Platform Blueprint**, un documento que detalle exactamente:

- La estructura final del repositorio.

- La base de datos definitiva en inglés.
- Los contratos entre motores.
- Los eventos del sistema.
- Las interfaces de repositorios.
- Los modelos de dominio.
- Los casos de uso.
- Las reglas de despliegue.
- Los estándares de pruebas.

Ese documento sería la referencia única para desarrollar toda la plataforma, evitando inconsistencias y permitiendo que BiteFixes crezca de forma ordenada durante muchos años.